

ALGORITHM ANALYSIS AND DESIGN

INTERMEDIATE LEVEL - DETAILED LEARNING NOTES

1. Deep Understanding of P vs NP

Class P contains decision problems solvable in polynomial time using deterministic algorithms. Class NP contains problems whose solutions can be verified in polynomial time. Important concept: Every P problem is in NP, but it is unknown whether $P = NP$. The P vs NP question asks whether every problem whose solution can be verified quickly can also be solved quickly.

2. Concept of Polynomial Time Reduction (Detailed)

Reduction is used to compare difficulty of problems. If problem A can be transformed into problem B in polynomial time, then B is at least as hard as A. Used heavily in NP-Completeness proofs.

3. General Framework for Proving NP-Completeness

Step 1: Show problem is in NP (verification in polynomial time). Step 2: Choose a known NP-Complete problem. Step 3: Reduce known problem to new problem in polynomial time. Step 4: Conclude NP-Hardness. If both NP and NP-Hard $\rightarrow$ NP-Complete.

4. Detailed Study of SAT Problem

Boolean Satisfiability (SAT) checks if a boolean formula has a satisfying assignment. It was the first problem proven NP-Complete (Cook's Theorem). All NP problems can be reduced to SAT.

5. Clique Problem – Detailed Explanation

Clique asks whether a graph contains a complete subgraph of size k . Verification requires checking $O(k^2)$ edges. SAT reduces to Clique, proving it NP-Hard.

6. Vertex Cover – Complement Graph Method

Vertex Cover asks for a set of vertices covering all edges. Reduction from Clique uses complement graph concept: Clique of size k in $G \leftrightarrow$ Vertex Cover of size $|V|-k$ in G' .

7. Hamiltonian Cycle Problem

Determines if a cycle exists visiting each vertex exactly once. Verification is polynomial, but finding such cycle is NP-Complete.

8. Optimization vs Decision Problems

Decision problems give Yes/No answer. Optimization problems find best solution. Many NP-Complete problems are decision versions of optimization problems.

9. Approximation Algorithms – Formal Understanding

Used when exact solutions are computationally infeasible. Guarantee approximation ratio $P(n) \geq 1$. Used for NP-Hard optimization problems like Bin Packing and Vertex Cover.

10. Approximation Ratio Analysis

For minimization: Ratio = $C / C^* \geq 1$. For maximization: Ratio = $C^* / C \geq 1$. Smaller ratio means better approximation.

11. Detailed Bin Packing Algorithms Comparison

Next Fit $\rightarrow O(n)$, Ratio ≤ 2 . First Fit $\rightarrow O(n^2)$ naive, Ratio ≤ 1.7 . Best Fit $\rightarrow$ Similar to First Fit. First Fit Decreasing $\rightarrow$ Sorting + First Fit, better performance.

12. First Fit Decreasing – Why It Works Better

Sorting large items first reduces wasted space. Guarantee: Uses at most $(4M+1)/3$ bins if optimal is M .

13. Graph Coloring – Backtracking Complexity

Assign colors recursively. State space size = M^n (M colors, n vertices). Hence exponential complexity $O(M^n)$.

14. Chromatic Number and Its Complexity

Chromatic number $\chi(G)$ is minimum colors required. Finding exact chromatic number is NP-Complete.

15. Edge Coloring vs Vertex Coloring

Vertex coloring colors nodes. Edge coloring colors edges such that no adjacent edges share color.

16. Randomized Algorithms – Mathematical View

Randomized algorithms use probability to improve expected performance. Performance measured using expected time complexity.

17. Las Vegas vs Monte Carlo – Formal Difference

Las Vegas $\rightarrow$ Always correct, random running time (expected analysis). Monte Carlo $\rightarrow$ Deterministic running time, small error probability.

18. Randomized QuickSort – Expected Analysis

Pivot chosen randomly. Worst-case avoided with high probability. Expected recurrence:
 $T(n) = T(n/4) + T(3n/4) + O(n)$. Expected time = $O(n \log n)$.

19. Lower Bound Theory

Lower bound gives theoretical minimum time to solve problem. If algorithm matches lower bound, it is optimal.

20. Applications of Complexity Theory

Used in cryptography, network design, scheduling, compiler optimization, resource allocation, and distributed systems.